

Metodologie Agili

Traduzione e Riassunto

1. Introduzione allo Sviluppo Rapido e Agile

- Lo sviluppo e la consegna rapida sono oggi i requisiti più importanti per i sistemi software, poiché le aziende operano in un ambiente in cui le esigenze cambiano velocemente.
- A differenza dello sviluppo tradizionale "basato sui piani" (plan-driven) – che richiede una pianificazione dettagliata di ogni singola fase prima di procedere alla successiva – lo sviluppo Agile intreccia le specifiche, la progettazione e l'implementazione.
- Nel modello Agile il software viene sviluppato in incrementi frequenti, con la collaborazione costante degli stakeholder, minimizzando la documentazione per concentrarsi sul codice funzionante.

2. Il Manifesto e i Principi Agile

- Il Manifesto Agile stabilisce che si debba dare più valore a: individui e interazioni (rispetto a processi e strumenti), software funzionante (rispetto alla documentazione onnicomprensiva), collaborazione con il cliente (rispetto alla negoziazione dei contratti) e capacità di rispondere al cambiamento (rispetto al seguire rigidamente un piano).
- I principi chiave includono il coinvolgimento continuo del cliente, le consegne incrementali, l'affidamento sulle competenze delle persone, l'accettazione del cambiamento e la ricerca della semplicità (eliminare la complessità dal sistema).

3. Extreme Programming (XP)

- L'Extreme Programming è un influente metodo Agile nato alla fine degli anni '90 che porta all'estremo lo sviluppo iterativo. Le versioni del software possono essere create anche più volte al giorno, con rilasci ai clienti ogni 2 settimane.
- **Pratiche fondamentali di XP:**
 - **Test-first development:** i test automatizzati vengono scritti prima di implementare il codice vero e proprio.
 - **Refactoring:** miglioramento e riorganizzazione costante del codice (es. rinominare variabili, rimuovere duplicati) per renderlo semplice e mantenibile, ancor prima che ci sia un bisogno immediato di farlo.
 - **Pair programming:** due sviluppatori lavorano insieme sullo stesso computer; questo funge da revisione informale e diffonde la conoscenza all'interno del team.
 - **Cliente sul posto:** un rappresentante degli utenti lavora a tempo pieno col team di sviluppo fornendo i requisiti.

4. Gestione Agile dei Progetti (Scrum)

- Scrum è un framework di gestione dei progetti che si concentra sull'organizzazione dello sviluppo iterativo piuttosto che sulle pratiche tecniche.
- **Ciclo di vita e Terminologia Scrum:**
 - **Sprint:** cicli di sviluppo di durata fissa (solitamente 2-4 settimane) alla fine dei quali viene consegnato un incremento di software potenzialmente utilizzabile e rilasciabile.
 - **Product Backlog:** un elenco di compiti e funzionalità prioritarie da sviluppare.
 - **Product Owner:** la persona incaricata di identificare, dare priorità ai requisiti e mantenere aggiornato il backlog in base alle esigenze aziendali.
 - **Scrum Master:** un facilitatore che guida il team nell'uso di Scrum e lo protegge dalle distrazioni e interferenze esterne.
 - **Scrum giornaliero (Daily Scrum):** una breve riunione quotidiana in cui il team condivide i progressi e pianifica il lavoro della giornata.

5. Scalabilità (Scaling Agile)

- I metodi Agile funzionano in modo eccellente per team piccoli e localizzati nello stesso luogo, ma risulta difficile "scalarli" (applicarli in grande) su sistemi software molto grandi o all'interno di organizzazioni complesse.
- **Sfide:** L'uso dell'Agile su larga scala incontra ostacoli legati ai contratti legali standard (che si basano su requisiti predefiniti), alla manutenzione di sistemi software datati ("brown-field systems"), alle regolamentazioni governative che esigono documentazione formale dettagliata, e al coordinamento di team distribuiti in diverse parti del mondo.
- **Soluzioni su larga scala:** Per i grandi progetti, si adattano le tecniche Agile tramite architetture di prodotto condivisi, l'allineamento delle scadenze di rilascio tra vari team e lo "Scrum of Scrums" (una riunione giornaliera tra i rappresentanti dei vari team).

Conclusioni

Nella pratica moderna, lo sviluppo di sistemi di grandi dimensioni richiede spesso un mix di metodologie basate sui piani (per l'architettura iniziale e la documentazione) e metodologie Agile (per lo sviluppo rapido del codice). La scelta ottimale dipende da fattori legati al sistema (scala, normative, tempo di vita), al team (competenza, distribuzione) e all'organizzazione (cultura, contratti).